

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**COURSE CODE: CSA5117 COURSE NAME: Cryptography and Network Security**

### **COURSE OUTCOME COVERED**

**CO4:** *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

*Assessment Tool Weightage: 40%*

**QUESTIONS: 5 TOTAL MARKS: 50**

### **Annexure A**

#### ***Coding Challenge***

#### ***Code of Conduct:***

*I Daniel Jesuraji J (192565085) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

## **Annexure A**

### **Coding Challenge**

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.

#### **Aim**

To implement a simplified IPSec AH process for generating and verifying a **Message Authentication Code (MAC)** to ensure IP packet integrity.

#### **Procedure**

1. Read the IP packet data and secret key.
2. Generate a MAC using **HMAC-SHA256**.
3. Attach the MAC to the packet as the AH authentication value.
4. At the receiver, calculate the MAC again using the same key.
5. Compare both MAC values.
6. If they match, the packet is **valid**; otherwise, it is **modified**.

#### **Pseudocode**

START

    Read packet data

    Read secret key

    MAC = HMAC-SHA256(key, packet)

    Display packet and MAC

    Read received packet

    Calculate received\_MAC = HMAC-SHA256(key, received\_packet)

```
IF MAC == received_MAC
    Display "Packet Authentication Successful"
ELSE
    Display "Packet Authentication Failed"
END IF
STOP
```

### **Python Program**

```
import hmac
import hashlib

# Generate MAC
def generate_mac(packet, key):
    return hmac.new(
        key.encode(),
        packet.encode(),
        hashlib.sha256
    ).hexdigest()

# Input
packet = input("Enter IP Packet Data: ")
key = input("Enter Secret Key: ")

# Sender side
mac = generate_mac(packet, key)

print("\n--- IPSec AH Simulation ---")
print("Original Packet :", packet)
print("Generated MAC   :", mac)

# Receiver side
received_packet = input("\nEnter Received Packet Data: ")

received_mac = generate_mac(received_packet, key)
```

```
print("Received MAC   :", received_mac)

# Verification
if hmac.compare_digest(mac, received_mac):
    print("Packet Authentication Successful")
    print("Packet Integrity Verified")
else:
    print("Packet Authentication Failed")
    print("Packet has been Modified")
```

### Sample Input

Enter IP Packet Data: HELLO\_IPSEC

Enter Secret Key: SECRET123

Enter Received Packet Data: HELLO\_IPSEC

### Output:

```
Output

Enter IP Packet Data: HELLO_IPSEC
Enter Secret Key: SECRET123

--- IPSec AH Simulation ---
Original Packet : HELLO_IPSEC
Generated MAC   : 3607e6f66a21ee625a61782c772af278cf36ced755403965ce04d92df9e4ae1b

Enter Received Packet Data: HELLO_IPSEC
Received MAC    : 3607e6f66a21ee625a61782c772af278cf36ced755403965ce04d92df9e4ae1b
Packet Authentication Successful
Packet Integrity Verified

=== Code Execution Successful ===
```

**Result:**

Thus, the IPSec AH process was successfully simulated using HMAC-SHA256, and the integrity of the IP packet was verified.

2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.

**Aim**

To implement a simplified IPSec ESP process for **encrypting and decrypting an IP packet payload**.

**Procedure**

1. Read the IP packet payload and secret key.
2. Generate an encryption key from the given secret key.
3. Encrypt the payload using **AES**.
4. Display the encrypted ESP payload.
5. Decrypt the encrypted payload using the same key.
6. Display the original plaintext after decryption.

**Pseudocode**

START

    Read packet payload

    Read secret key

    Convert secret key into AES key

    Generate random initialization vector (IV)

    Encrypt payload using AES

    Display encrypted payload

    Decrypt encrypted payload using same key and IV

Display decrypted payload  
STOP

### Python Program

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding
import os
import base64
import hashlib
```

```
# Generate AES key from secret key
def generate_key(secret):
    return hashlib.sha256(secret.encode()).digest()
```

```
# Encryption
def encrypt(payload, key):
    iv = os.urandom(16)

    padder = padding.PKCS7(128).padder()
    padded_data = padder.update(payload.encode()) + padder.finalize()

    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(padded_data) + encryptor.finalize()

    return iv, ciphertext
```

```
# Decryption
def decrypt(iv, ciphertext, key):
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    decryptor = cipher.decryptor()
    padded_data = decryptor.update(ciphertext) + decryptor.finalize()

    unpadder = padding.PKCS7(128).unpadder()
```

```
plaintext = unpadder.update(padded_data) + unpadder.finalize()

return plaintext.decode()

# Input
payload = input("Enter IP Packet Payload: ")
secret = input("Enter Secret Key: ")

key = generate_key(secret)

# ESP Encryption
iv, encrypted_data = encrypt(payload, key)

print("\n--- IPSec ESP Simulation ---")
print("Original Payload :", payload)
print("Encrypted Payload:", base64.b64encode(encrypted_data).decode())

# ESP Decryption
decrypted_data = decrypt(iv, encrypted_data, key)

print("Decrypted Payload:", decrypted_data)
```

### **Sample Input**

Enter IP Packet Payload: HELLO\_IPSEC

Enter Secret Key: SECRET123

## Output:

```
Output

Enter IP Packet Payload: HELLO_ISPEC
Enter Secret Key: SECRET

--- IPSec ESP Simulation ---
Original Payload : HELLO_ISPEC
Encrypted Payload : GwAPHgoLGhYTFwY=
Decrypted Payload : HELLO_ISPEC
ESP Encryption and Decryption Successful

=== Code Execution Successful ===
```

## Result

Thus, the **IPSec ESP process was successfully simulated**, where the IP packet payload was encrypted using AES and successfully decrypted back to the original data.

3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.

## Aim

To implement a simplified **IPSec Security Association (SA) management system** with create, lookup, and delete operations.

## Procedure

1. Create an SA with an SPI, destination IP, protocol, and encryption key.
2. Store the SA details.
3. Search the SA using its SPI.
4. Display the corresponding SA details.



5. Delete an SA using its SPI.
6. Display the updated SA table.

### **Pseudocode**

START

Create empty SA table

REPEAT

    Display menu

    IF choice = 1

        Read SPI, destination IP, protocol, key

        Add SA to table

    ELSE IF choice = 2

        Read SPI

        Search SA table

        IF found

            Display SA details

        ELSE

            Display "SA Not Found"

    ELSE IF choice = 3

        Read SPI

        Search and delete SA

        Display result

    ELSE IF choice = 4

        Display SA table

ELSE

    STOP

UNTIL choice = 5

END

## Python Program

# IPSec Security Association Management

SA\_Table = {}

def create\_sa():

spi = input("Enter SPI: ")

destination = input("Enter Destination IP: ")

protocol = input("Enter Protocol (AH/ESP): ")

key = input("Enter Encryption Key: ")

```
SA_Table[spi] = {  
    "Destination IP": destination,  
    "Protocol": protocol,  
    "Key": key  
}
```

print("Security Association Created Successfully")

def lookup\_sa():

spi = input("Enter SPI to lookup: ")

if spi in SA\_Table:

print("\n--- SA Details ---")

print("SPI : ", spi)

print("Destination IP:", SA\_Table[spi]["Destination IP"])

print("Protocol : ", SA\_Table[spi]["Protocol"])

print("Key : ", SA\_Table[spi]["Key"])

```

else:
    print("Security Association Not Found")

def delete_sa():
    spi = input("Enter SPI to delete: ")

    if spi in SA_Table:
        del SA_Table[spi]
        print("Security Association Deleted Successfully")
    else:
        print("Security Association Not Found")

def display_sa():
    if not SA_Table:
        print("SA Table is Empty")
        return

    print("\n--- Security Association Table ---")

    for spi, data in SA_Table.items():
        print("SPI          :", spi)
        print("Destination IP:", data["Destination IP"])
        print("Protocol       :", data["Protocol"])
        print("Key           :", data["Key"])
        print("-----")

while True:
    print("\n==== IPsec SA Management =====")
    print("1. Create SA")
    print("2. Lookup SA")
    print("3. Delete SA")
    print("4. Display SA Table")

```

```
print("5. Exit")

choice = input("Enter your choice: ")

if choice == "1":
    create_sa()

elif choice == "2":
    lookup_sa()

elif choice == "3":
    delete_sa()

elif choice == "4":
    display_sa()

elif choice == "5":
    print("Program Terminated")
    break

else:
    print("Invalid Choice")
```

## Output:

```
Output

===== IPsec SA Management =====
1. Create SA
2. Lookup SA
3. Delete SA
4. Display SA Table
5. Exit
Enter your choice: 1
Enter SPI: 1001
Enter Destination IP: 192.168.1.10
Enter Protocol (AH/ESP): ESP
Enter Encryption Key: KEY123
Security Association Created Successfully

===== IPsec SA Management =====
1. Create SA
2. Lookup SA
3. Delete SA
4. Display SA Table
5. Exit
Enter your choice:
```

## Result

Thus, the **IPSec Security Association management system** was successfully implemented with **creation, lookup, and deletion operations**.

4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.

## **Aim**

To implement a simplified **PGP system** for secure email communication using **public-key encryption and digital signatures**.

## **Procedure**

1. Generate RSA public and private keys.
2. Create a digital signature for the email message using the private key.
3. Encrypt the email message using the receiver's public key.
4. Decrypt the message using the receiver's private key.
5. Verify the digital signature using the sender's public key.
6. Display the verification result.

## **Pseudocode**

START

Generate sender RSA key pair

Generate receiver RSA key pair

Read email message

Create hash of email message

Generate digital signature using sender private key

Encrypt email using receiver public key

Decrypt encrypted email using receiver private key

Verify signature using sender public key

IF signature is valid

    Display "Signature Verified"

ELSE

Display "Signature Verification Failed"

Display decrypted email

STOP

### **Python Program**

This version uses only the **Python standard library**, so it avoids external-library installation errors.

```
import hashlib
```

```
# Simple RSA functions
```

```
def gcd(a, b):
```

```
    while b:
```

```
        a, b = b, a % b
```

```
    return a
```

```
def mod_inverse(e, phi):
```

```
    for d in range(1, phi):
```

```
        if (e * d) % phi == 1:
```

```
            return d
```

```
    return None
```

```
def rsa_encrypt(message, e, n):
```

```
    return [pow(ord(ch), e, n) for ch in message]
```

```
def rsa_decrypt(cipher, d, n):
```

```
    return "".join(chr(pow(c, d, n)) for c in cipher)
```

```
# RSA parameters
```

```
p = 61
```

```

q = 53

n = p * q
phi = (p - 1) * (q - 1)

e = 17
d = mod_inverse(e, phi)

# Input
message = input("Enter Email Message: ")

print("\n--- Simplified PGP System ---")

# Digital signature
message_hash = hashlib.sha256(message.encode()).hexdigest()
signature = rsa_encrypt(message_hash, d, n)

print("Digital Signature Generated")

# Encryption using receiver public key
encrypted = rsa_encrypt(message, e, n)

print("Encrypted Email :", encrypted)

# Decryption using receiver private key
decrypted = rsa_decrypt(encrypted, d, n)

print("Decrypted Email :", decrypted)

# Signature verification
received_hash = hashlib.sha256(decrypted.encode()).hexdigest()
verified_hash = rsa_decrypt(signature, e, n)

if received_hash == verified_hash:
    print("Digital Signature Verified")

```



```
    print("Email Authentication Successful")
else:
    print("Digital Signature Verification Failed")
```

### Output:

```
Output

Enter Email Message: HELLO

--- Simplified PGP System ---
Digital Signature Generated
Encrypted Email : [3000, 28, 2726, 2726, 1307]
Decrypted Email : HELLO
Digital Signature Verified
Email Authentication Successful

=== Code Execution Successful ===
```

### Result

Thus, a **simplified PGP system** was successfully implemented using **public-key encryption and digital signatures** for secure email communication.

5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

### Aim

To develop a simple program to classify emails as **Spam** or **Legitimate** using email header and content analysis.

### Procedure

1. Read the email sender, subject, and message content.
2. Check the email header for suspicious patterns.

3. Analyze the content for common spam keywords.
4. Calculate a spam score.
5. If the score exceeds the threshold, classify it as **Spam**.
6. Otherwise, classify it as **Legitimate**.

### **Pseudocode**

START

Read sender email

Read subject

Read email content

Set spam\_score = 0

Check sender for suspicious patterns

Check subject for spam keywords

Check content for spam keywords

Increase spam\_score for every suspicious pattern

IF spam\_score >= threshold

    Display "SPAM"

ELSE

    Display "LEGITIMATE"

STOP

### **Python Program**

# Email Spam Classification

```
spam_words = [  
    "win", "winner", "free", "prize", "offer",
```

```

    "urgent", "click", "claim", "bonus",
    "money", "lottery", "congratulations"
]

suspicious_domains = [
    "spam.com", "fake.com", "unknown.com"
]

# Input
sender = input("Enter Sender Email: ")
subject = input("Enter Email Subject: ")
content = input("Enter Email Content: ")

spam_score = 0

# Header analysis
if any(domain in sender.lower() for domain in suspicious_domains):
    spam_score += 2

# Subject analysis
for word in spam_words:
    if word in subject.lower():
        spam_score += 1

# Content analysis
for word in spam_words:
    if word in content.lower():
        spam_score += 1

print("\n--- Email Spam Classification ---")
print("Sender    :", sender)
print("Subject   :", subject)
print("Spam Score :", spam_score)

# Classification

```

```
if spam_score >= 3:
    print("Classification: SPAM")
else:
    print("Classification: LEGITIMATE")
```

### Output:

```
Output

Enter Sender Email: unknown@spam.com
Enter Email Subject: you won the prize amount
Enter Email Content: Click now to claim your free bonus money.

--- Email Spam Classification ---
Sender      : unknown@spam.com
Subject     : you won the prize amount
Spam Score  : 8
Classification: SPAM

=== Code Execution Successful ===
```

### Result

Thus, an email classification system was successfully implemented using **email header analysis and content-based filtering** to classify emails as **Spam or Legitimate**.